Benyamin Gidanian

Los Angeles, CA 90034 · (310) 613-3655 · bgidanian@gmail.com
GitHub · LinkedIn

Summary

Full-stack and applied ML engineer who builds and ships systems end-to-end — from RESTful APIs and React frontends to fine-tuned ML models in production-ready services. Comfortable across the stack: Python backend services (FastAPI, Django), React/JavaScript frontends, Docker containerization, and CI/CD. Background spans NLP (transformer fine-tuning, token classification), computer vision (transfer learning, class imbalance), and full-stack web development. Strong with AI-assisted development workflows. Moves fast, experiments deliberately, and focuses on shipping things that work.

Technical Skills

Languages: Python, JavaScript, SQL, C
Frontend: React, JavaScript, HTML, CSS
Backend / APIs: FastAPI, Django, RESTful API design, SQL (SQLite)
ML/AI: PyTorch, TensorFlow, Keras, Hugging Face Transformers, BERT, ConvNeXt, timm, CUDA
ML Engineering: Transfer Learning, Hyperparameter Tuning, Model Evaluation (ROC/AUC, F1, Precision/Recall), Weights & Biases
Infra & DevOps: Docker, GitHub Actions (CI/CD), Linux (Arch), Bash, Git
Data: NumPy, Pandas, Feature Engineering, Data Preprocessing
Tools: HuggingFace Hub, Pytest, Ruff, Pyright, uv, Jupyter Notebooks

---

Projects

PII Redaction

PyTorch · DeBERTa-v3 · HuggingFace Trainer/Hub · FastAPI · CLI GitHub

Models: base · small · xsmall

- Fine-tuned three DeBERTa-v3 models on `ai4privacy/pii-masking-300k` for PII named entity recognition across 27 entity types (55 BIO labels: names, emails, SSNs, phone numbers, etc.), achieving macro F1 of **0.9557 (base)**, **0.9517 (small)**, and **0.9424 (xsmall)** on the held-out test set.
- Built a custom `WeightedTokenClassificationTrainer` with class-balanced cross-entropy loss to handle severe O-token imbalance; trained in two phases (frozen backbone → full fine-tune) using bf16 mixed precision, with W&B logging including per-entity F1 and interactive HTML prediction tables.
- Implemented a `PIIRedactor` class with overlapping token chunking (paragraph/sentence-aware boundary alignment), overlap resolution by max confidence score, and structured Pydantic `Entity` / `RedactionResponse` schemas; exposed via a CLI supporting `--text`, `--file`, threshold tuning, stride/max-length overrides, and JSON output.
- Built a FastAPI service with API-key-authenticated `POST /redact`, `GET /health`, `GET /model-info`, and an interactive dark-mode `GET /demo` with glassmorphism layout, drag-and-drop file upload, and confidence badges; `POST /demo/redact` is unauthenticated but rate-limited (10/day per IP via slowapi). 14 unit tests mock the redactor across auth, rate limiting, validation, and edge cases.
- Published all three model checkpoints to HuggingFace Hub with full model cards documenting per-model F1, GPU latency benchmarks, distribution-shift behavior, and per-entity tradeoffs.

Network — Full-Stack Social Platform

Django · React · JavaScript · SQLite
GitHub

Social Network — A Django social platform with React-powered inline editing and async likes. Features user auth, posts, follow system, like/unlike, profile pages, and a following feed — all with pagination. Uses Django for routing/auth/data and vanilla React (CDN) only for interactive components, demonstrating pragmatic hybrid architecture. Explicit through-models (Like, Follow) with unique_together constraints prevent duplicates at the DB level.

Flower Classification Inference Service

PyTorch · EfficientNet · FastAPI · Docker · HuggingFace Hub · GitHub Actions
GitHub · Model

- Achieved **>93% test accuracy** on the 102-class Oxford Flowers dataset via two-stage transfer learning of EfficientNet-B0 (classifier-only warm-up, then partial backbone unfreeze with differential learning rates).
- Served the model via a FastAPI service (`/predict` , `/health`) with file-type, size, and image-dimension validation; integrated automatic weight download from HuggingFace Hub on cold start.
- Containerized with Docker with a full CI pipeline (GitHub Actions): Ruff, Pyright, Pytest (model-mocked for GPU-free CI), and Docker build verification on every push.

Gravitational Lens Finding — ML4SCI DeepLense (GSoC 2026 Evaluation)

PyTorch · ConvNeXt-Tiny · EfficientNet-B0 · timm
GitHub

- Achieved **0.9887 AUROC** on binary gravitational lens classification (16.6:1 class imbalance) using ConvNeXt-Tiny, and **0.98 macro AUROC** on 3-class substructure classification via two-phase fine-tuning of EfficientNet-B0.
- Addressed severe class imbalance with combined `WeightedRandomSampler` oversampling and calibrated `BCEWithLogitsLoss` pos-weighting.

Additional Projects

- **ML Fundamentals Library** *(Python, NumPy, Pytest)* — Built a from-scratch ML library (Linear/Logistic Regression, K-Means, Decision Trees, multi-layer NN with manual backprop) with 100% test coverage and a custom AST-based doc generator. GitHub
- **DQN for Autonomous Lunar Landing** *(TensorFlow, Keras, Gymnasium)* — Implemented a DQN agent with experience replay and soft-update target networks, solving LunarLander-v3 with a rolling average score above 200. GitHub
- **Crossword Generator** *(Python, CSP/AC-3)* — Built a constraint satisfaction solver using AC-3 arc consistency plus MRV/LCV heuristics, reducing backtracking search space by over 80%. GitHub

---

Professional Experience

One-on-One Talmud Tutor

Mesivta Bircas Yitzchok · Sept 2023 – March 2025

- Delivered individualized instruction over a 1.5-year engagement, developing structured curricula to accelerate comprehension and long-term retention.

Education & Certifications

Machine Learning Specialization — Stanford University & DeepLearning.AI (Coursera) Supervised Learning, Neural Networks, Decision Trees, Unsupervised Learning, Recommender Systems

PyTorch for Deep Learning (Professional Certificate) — DeepLearning.AI
Neural Networks & Tensors, CNNs, Transfer Learning & Fine-Tuning, NLP with Transformers & Hugging Face, Advanced Architectures (ResNet, DenseNet, Siamese Networks, multi-head attention), Model Interpretability, ONNX Export, Pruning & Quantization, Experiment Tracking (MLflow)

Harvard CS50 Suite — Harvard University (EdX)
CS50x (C, Python, SQL, Algorithms) · CS50p (OOP, Testing) · CS50ai (Search, Logic, Optimization) · CS50w (Django, React, CI/CD)